

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: ITA 0614

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO5: *Introduction – NumPy Arrays – NumPy Basics- Math – Random – Indexing – Filtering – Statistics – Aggregation – saving data – Data analysis with Pandas – DataFrame – Combining – Indexing – File I/O – Grouping – Filtering – Sorting – Plotting*
(BL 3)

Assessment Tool Weightage: 40%

QUESTIONS: 4

Total Marks:40

Annexure A

Constraint Based Problem Solving

Code of Conduct:

*I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Annexure A

Short Answer Test

```
1, import pandas as pd
import numpy as np

# Create sample patient dataset
data = {
    "Age": [25, 45, np.nan, 67, 150, 32, 54, 29],
    "Weight": [65, 80, 70, np.nan, 400, 55, 90, 68],
    "BMI": [22.5, 27.8, 24.1, 31.2, 150, np.nan, 29.5, 23.4],
    "Heart_Rate": [72, 80, 75, 90, 300, np.nan, 85, 70],
    "Gender": ["Male", "male", "FEMALE", "Female", np.nan, "F", "M",
"female"],
    "Diabetes": ["Yes", "No", "yes", "NO", "Yes", np.nan, "Y", "N"]
}

df = pd.DataFrame(data)

# Standardize categorical values
df["Gender"] = df["Gender"].str.lower().replace({
    "m": "male", "f": "female"
})
df["Diabetes"] = df["Diabetes"].str.lower().replace({
    "y": "yes", "n": "no"
})

# Fill missing values
num = df.select_dtypes(include=np.number).columns
cat = df.select_dtypes(include="object").columns

df[num] = df[num].fillna(df[num].median())
for c in cat:
    df[c] = df[c].fillna(df[c].mode()[0])

# Apply medical constraints
df["Age"] = df["Age"].clip(0, 120)
df["Weight"] = df["Weight"].clip(1, 300)
df["BMI"] = df["BMI"].clip(5, 100)
df["Heart_Rate"] = df["Heart_Rate"].clip(20, 250)
```

```

# Indexing and filtering
print("Patients above age 50:")
print(df.loc[df["Age"] > 50])

# Aggregation
print("\nStatistics:")
print(df[num].agg(["min", "max", "mean"]))

# Verify missing values
print("\nMissing values:", df.isnull().sum().sum())

print("\nCleaned Dataset:")
print(df)

```

Patients above age 50:

	Age	Weight	BMI	Heart_Rate	Gender	Diabetes
3	67.0	70.0	31.2	90.0	female	no
4	120.0	300.0	100.0	250.0	female	yes
6	54.0	90.0	29.5	85.0	male	yes

Statistics:

	Age	Weight	BMI	Heart_Rate
min	25.000	55.00	22.5000	70.00
max	120.000	300.00	100.0000	250.00
mean	52.125	99.75	35.7875	100.25

Missing values: 0

Cleaned Dataset:

	Age	Weight	BMI	Heart_Rate	Gender	Diabetes
0	25.0	65.0	22.5	72.0	male	yes
1	45.0	80.0	27.8	80.0	male	no
2	45.0	70.0	24.1	75.0	female	yes
3	67.0	70.0	31.2	90.0	female	no
4	120.0	300.0	100.0	250.0	female	yes
5	32.0	55.0	27.8	80.0	female	yes
6	54.0	90.0	29.5	85.0	male	yes
7	29.0	68.0	23.4	70.0	female	no

3. # Small Google Colab Code: Combining Customer DataFrames

```

import pandas as pd
import numpy as np

# Create transaction dataset
transactions = pd.DataFrame({
    "Customer_ID": [101, 102, 103, 104, 105],
    "Amount": [500, 1200, 750, 300, 1500]
})

```

```

}))

# Create demographic dataset
demographics = pd.DataFrame({
    "Customer_ID": [101, 102, 103, 104, 106],
    "Age": [25, 35, 28, 42, 30],
    "City": ["Chennai", "Bangalore", "Hyderabad", "Chennai", "Mumbai"]
})

# Remove duplicate customer IDs
transactions = transactions.drop_duplicates("Customer_ID")
demographics = demographics.drop_duplicates("Customer_ID")

# Merge while preserving all valid records
df = pd.merge(
    transactions,
    demographics,
    on="Customer_ID",
    how="outer",
    indicator=True
)

# Sort and reset index
df = df.sort_values("Customer_ID").reset_index(drop=True)

# Check duplicate IDs
print("Duplicate IDs:")
print(df[df["Customer_ID"].duplicated(keep=False)])

# Check missing/mismatched records
print("\nMismatched Records:")
print(df[df["_merge"] != "both"])

# Grouping validation
print("\nCustomer ID Count:")
print(df.groupby("Customer_ID").size())

# Final index check
print("\nIndex:", df.index)
print("\nFinal Dataset:")
print(df)

# Remove merge indicator

```

```
df = df.drop(columns="_merge")
Duplicate IDs:
Empty DataFrame
Columns: [Customer_ID, Amount, Age, City, _merge]
Index: []
```

```
Mismatched Records:
  Customer_ID  Amount  Age  City  _merge
4          105  1500.0  NaN   NaN  left_only
5          106    NaN  30.0  Mumbai  right_only
```

Customer ID Count:

```
Customer_ID
101      1
102      1
103      1
104      1
105      1
106      1
dtype: int64
```

```
Index: RangeIndex(start=0, stop=6, step=1)
```

Final Dataset:

```
  Customer_ID  Amount  Age  City  _merge
0          101   500.0  25.0  Chennai  both
1          102  1200.0  35.0  Bangalore  both
2          103   750.0  28.0  Hyderabad  both
3          104   300.0  42.0  Chennai  both
4          105  1500.0   NaN   NaN  left_only
5          106    NaN  30.0  Mumbai  right_only
```

4. `import pandas as pd`

`import numpy as np`

`# Sample dataset`

```
df = pd.DataFrame({
    "Score": [10, 20, 30, 40, 50, 1000, np.nan]
})
```

`# Remove missing and irrelevant/outlier values`

```
df = df.dropna()
df = df[df["Score"] < 100]
```

`# Statistical measures`

```
mean = np.mean(df["Score"])
median = np.median(df["Score"])
variance = np.var(df["Score"])
```

```
print("Mean:", mean)
```

```

print("Median:", median)
print("Variance:", variance)

# Aggregation using Pandas
print("\nSummary:")
print(df["Score"].agg(["mean", "median", "var"]))

# Debug check
print("\nMissing values:", df["Score"].isnull().sum())
print("Number of records:", len(df))
Mean: 30.0
Median: 30.0
Variance: 200.0

Summary:
mean      30.0
median    30.0
var       250.0
Name: Score, dtype: float64

Missing values: 0
Number of records: 5

```

```

5. import pandas as pd
import matplotlib.pyplot as plt

# Create sample data files
pd.DataFrame({
    "Category": ["A", "B", "A", "C"],
    "Sales": [200, 150, 300, 100]
}).to_csv("data1.csv", index=False)

pd.DataFrame({
    "Category": ["B", "C", "A", "B"],
    "Sales": [250, 180, 400, 120]
}).to_csv("data2.csv", index=False)

# Load multiple files
files = ["data1.csv", "data2.csv"]
df = pd.concat([pd.read_csv(f) for f in files], ignore_index=True)

# Clean data
df = df.drop_duplicates().dropna()

# Filter

```

```

df = df[df["Sales"] > 100]

# Group and sort
result
df.groupby("Category")["Sales"].sum().sort_values(ascending=False)

print("Result:")
print(result)

# Plot
result.plot(kind="bar")
plt.xlabel("Category")
plt.ylabel("Total Sales")
plt.title("Sales by Category")
plt.show()

```

Result:

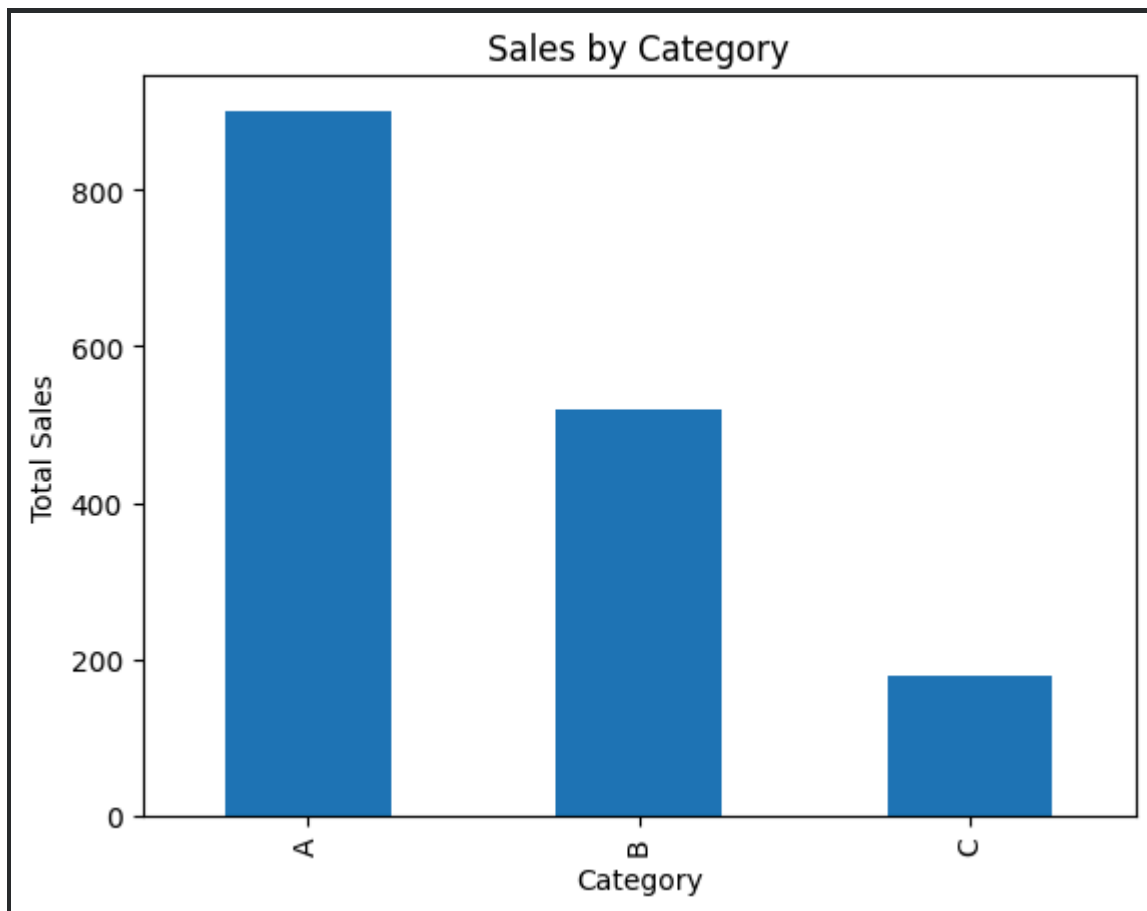
Category

A 900

B 520

C 180

Name: Sales, dtype: int64



RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	15	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand